

WEEK 6 – WEB SECURITY & VULNERABILITY TESTING

VAPT Basics — DVWA, Burp Suite and OWASP ZAP

Student: PAILACHUKKA SUSMITHA

DG Interns Hub – Cyber Security Internship

Target: Ubuntu Linux (DVWA) — 192.168.56.101:4280

Security Testing Machine: Kali Linux

Tools: DVWA | Burp Suite Community Edition | OWASP ZAP

Table of Contents

- 1. Introduction
- 2. Week 6 Objective
- 3. Lab Environment
- 4. OWASP Top 10 – Studied Concepts
- 5. DVWA Setup
- 6. SQL Injection Testing
- 7. Reflected XSS
- 8. Stored XSS
- 9. Burp Suite Testing
- 10. OWASP ZAP Automated Scan
- 11. Confirmed ZAP Findings
- 12. Vulnerability Findings Table
- 13. Risk Level Explanation
- 14. Mitigation and Security Recommendations
- 15. Challenges and Solutions
- 16. Learning Outcomes
- 17. Conclusion
- 18. References
- 19. Appendix

1. Introduction

This report documents Week 6 of the DG Interns Hub Cyber Security Internship, focused on the basics of Vulnerability Assessment and Penetration Testing (VAPT) for web applications. The practical work was carried out against the Damn Vulnerable Web Application (DVWA), an intentionally vulnerable application designed for security training, using Burp Suite Community Edition and OWASP ZAP as the primary testing tools.

All practical testing described in this report was performed only against an authorized local lab environment owned and operated by the student. No public or third-party systems were tested.

2. Week 6 Objective

The objective of this week was to understand common web application vulnerabilities and perform basic vulnerability testing using DVWA, Burp Suite Community Edition, and OWASP ZAP. The practical work included:

- OWASP Top 10 study
- DVWA setup
- SQL Injection testing
- Reflected XSS testing
- Stored XSS testing
- HTTP request/response analysis using Burp Suite
- Automated vulnerability scanning using OWASP ZAP
- Vulnerability finding analysis
- Risk and confidence analysis

- Basic mitigation recommendations

PAILACHUKKA SUSMITHA

3. Lab Environment

Component	Configuration
Host OS	Windows 11
Virtualization	Oracle VirtualBox
Security Testing Machine	Kali Linux
Target Machine	Ubuntu Linux
Target IP	192.168.56.101
DVWA Port	4280
DVWA URL	http://192.168.56.101:4280
Deployment Method	Docker (DVWA deployed on Ubuntu)
DVWA Security Level	Low

Tools Used

Tool	Purpose
Kali Linux	Security testing environment
Ubuntu Linux	Host machine for DVWA
Docker	Deploy and run DVWA
DVWA	Intentionally vulnerable web application used for security testing
Burp Suite Community Edition	HTTP proxy, request interception, request/response analysis, XSS request analysis
OWASP ZAP	Automated web vulnerability scanning, alert generation, risk analysis, security report generation

4. OWASP Top 10 – Studied Concepts

Before performing the practical testing, the following OWASP Top 10 concepts were studied to understand the vulnerabilities targeted during this week's assessment.

SQL Injection

SQL Injection occurs when untrusted user input is inserted directly into a SQL query without proper validation or parameterization, allowing an attacker to alter the intended logic of the query. It typically occurs when an application concatenates user input directly into a SQL statement. The impact can range from unauthorized data disclosure to modification or deletion of database records, depending on the privileges of the database account in use. Prevention relies on using prepared statements and parameterized queries, validating and sanitizing input, and applying the principle of least privilege to database accounts.

Cross-Site Scripting (XSS)

Cross-Site Scripting occurs when an application includes untrusted data in a web page without proper validation or escaping, allowing an attacker to run scripts in another user's browser.

Reflected XSS: The malicious script is included in a request (for example, a URL parameter) and is immediately reflected back in the application's response, executing in the victim's browser without being stored on the server.

Stored XSS: The malicious script is saved by the application (for example, in a database) and is later served to other users who view the affected page, without any further action from the attacker.

The impact of XSS can include session hijacking, credential theft, defacement, or redirection to malicious sites. Prevention relies on output encoding, input validation, Content Security Policy, and context-aware escaping.

Broken Authentication

Broken Authentication refers to weaknesses in an application's authentication and session management that allow attackers to compromise passwords, keys, or session tokens, or to exploit other implementation flaws to assume other users' identities. Common causes include weak password policies, predictable session identifiers, and improper session expiration. The impact can range from individual account compromise to full application takeover. Prevention includes strong authentication mechanisms, secure session management, multi-factor authentication, and proper session expiration and invalidation.

Security Misconfiguration

Security Misconfiguration occurs when security settings are not defined, implemented, or maintained correctly, or when default configurations are left unchanged. Examples include unnecessary features being enabled, default accounts and passwords remaining active, verbose error messages revealing internal details, and missing security headers. The impact depends on the specific misconfiguration but can expose sensitive information or provide an easier path for further attacks. Prevention involves applying secure configuration baselines, disabling unnecessary features, keeping software updated, and performing regular configuration reviews.

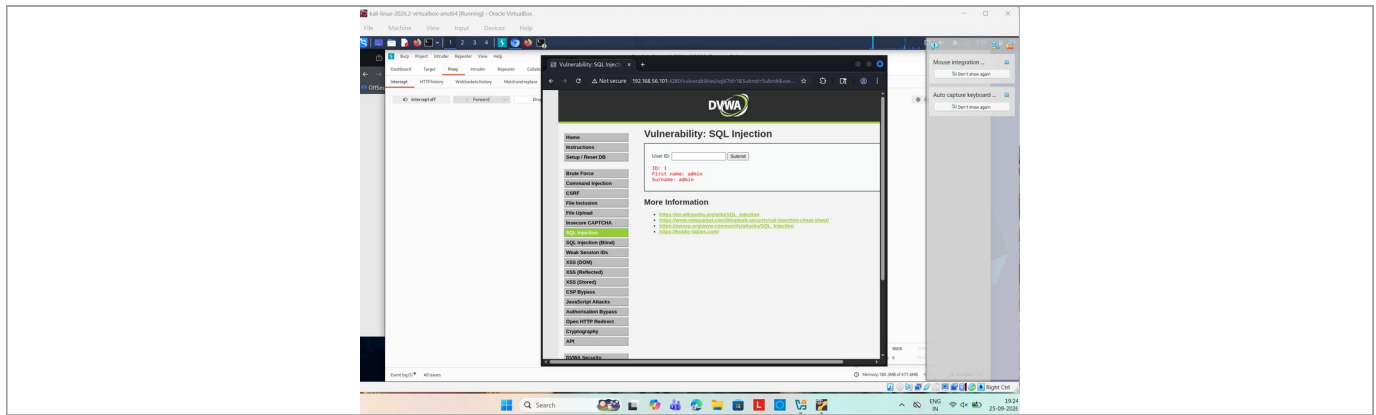


Figure 3: SQL Injection baseline test viewed through the Burp Suite browser, confirming the same normal result.

SQL Injection Payload

Payload used: 1' OR '1'='1 #

At a high level, this payload closes the original quoted value early and adds a condition ('1'='1') that is always true, which is intended to make the underlying query return more rows than the application's original, intended single-row lookup.

Observed result: submitting this payload into the User ID field returned multiple user records (admin, Gordon Brown, Hack Me, Pablo Picasso, and Bob Smith) instead of the single record expected from a normal lookup, confirming that the input was being interpreted as part of the SQL query rather than treated as a plain value.

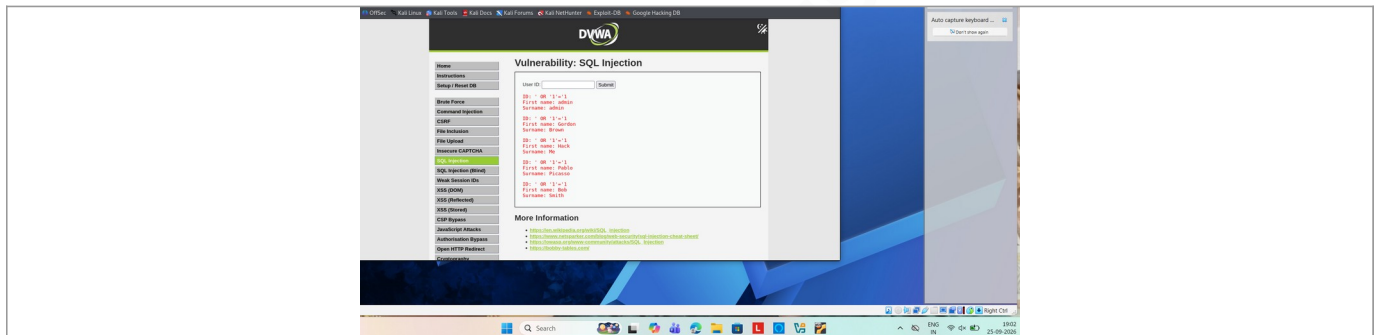


Figure 4: SQL Injection payload and application response, showing multiple user records returned.

Security Impact

This behaviour demonstrates that the application does not properly separate user-supplied input from the SQL query logic. In a real-world application, this class of vulnerability could allow an attacker to view data belonging to other users, bypass authentication logic, or in more severe cases, modify or delete database records.

Mitigation

- Use prepared statements.
- Use parameterized queries.
- Apply input validation.
- Use least-privilege database accounts.
- Avoid dynamic SQL construction.

7. Reflected XSS

Reflected Cross-Site Scripting occurs when user-supplied input is included in the application's response without proper validation or encoding, causing a script to execute immediately in the victim's browser as part of that response. It typically occurs through a URL parameter or form field whose value is echoed back into the page.

Payload used: `<script>alert('XSS')</script>`

This payload was submitted through the DVWA Reflected XSS page's "name" parameter. The application reflected the payload back into the page without sanitizing it, causing the script to execute and display a JavaScript alert box reading "XSS" — confirming that the payload was executed by the browser rather than displayed as plain text.

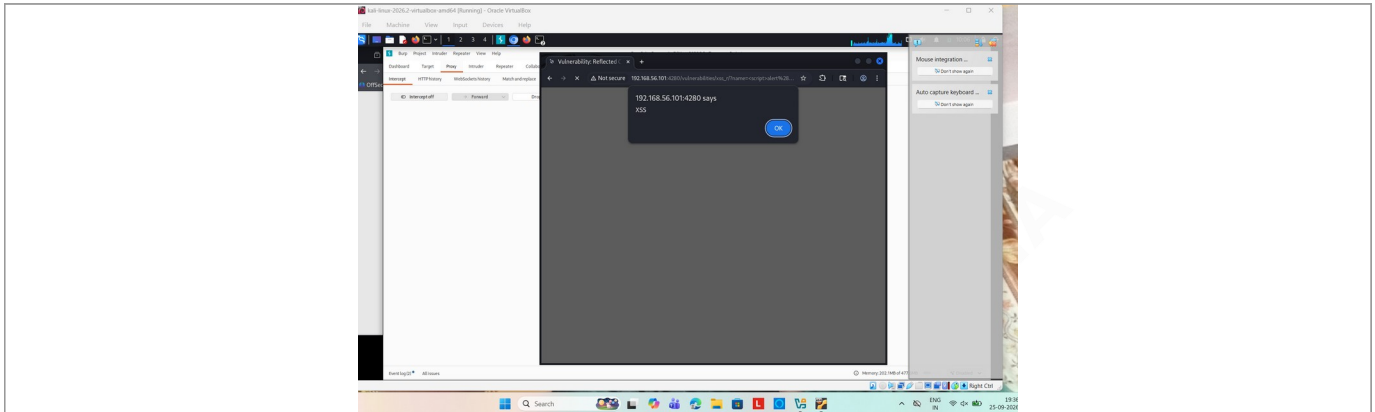


Figure 5: Reflected XSS successfully demonstrated in DVWA.

Security Impact

Reflected XSS can be used to execute arbitrary JavaScript in a victim's browser session, typically by tricking the victim into clicking a crafted link. This can lead to session hijacking, credential theft, or other actions performed in the context of the victim's session.

Mitigation

- Output encoding.
- Input validation.
- Content Security Policy.
- Context-aware escaping.
- Secure development practices.

8. Stored XSS

Stored Cross-Site Scripting occurs when a malicious script is saved by the application — for example, in a database — and is later served to other users who view the affected page. Unlike Reflected XSS, the payload does not need to be included in every request; it persists on the server and is served to any user who views the affected content, making it generally more dangerous.

Test data — Name: TestUser

Test data — Message: `<script>alert('Stored XSS')</script>`

This data was submitted through the DVWA Stored XSS guestbook form. When the page was reloaded, the stored message executed as a script rather than being displayed as text, producing a JavaScript alert box reading "Stored XSS" — confirming that the payload had been saved and was executing for any visitor to the page.

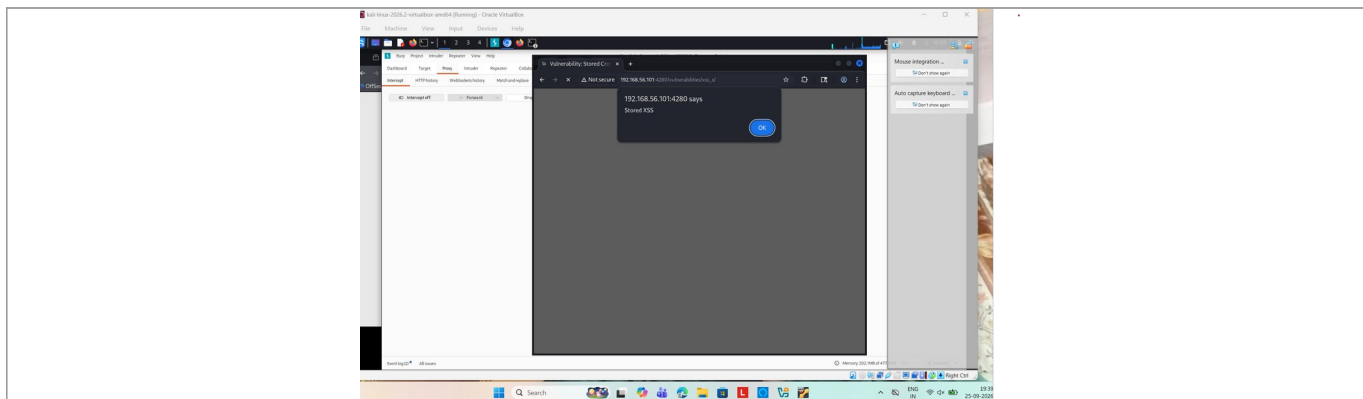


Figure 6: Stored XSS test performed in DVWA.

Security Impact

Because the payload is stored and served automatically, Stored XSS can affect every user who views the compromised page without requiring them to click a crafted link, making it a persistent and broader-reaching risk than Reflected XSS.

Mitigation

- Output encoding.
- Input validation.
- Content Security Policy.
- Context-aware escaping.
- Secure application development.

9. Burp Suite Testing

Burp Suite is a web application security testing platform used to intercept, inspect, and analyse HTTP traffic between a browser and a web application. It works as an HTTP proxy: the browser is configured to route its traffic through Burp, which allows each request and response to be viewed and examined before it is forwarded.

Burp proxy used: 127.0.0.1:8080

Burp Suite was used alongside DVWA testing to capture and analyse the HTTP requests generated during the SQL Injection and Reflected XSS tests.

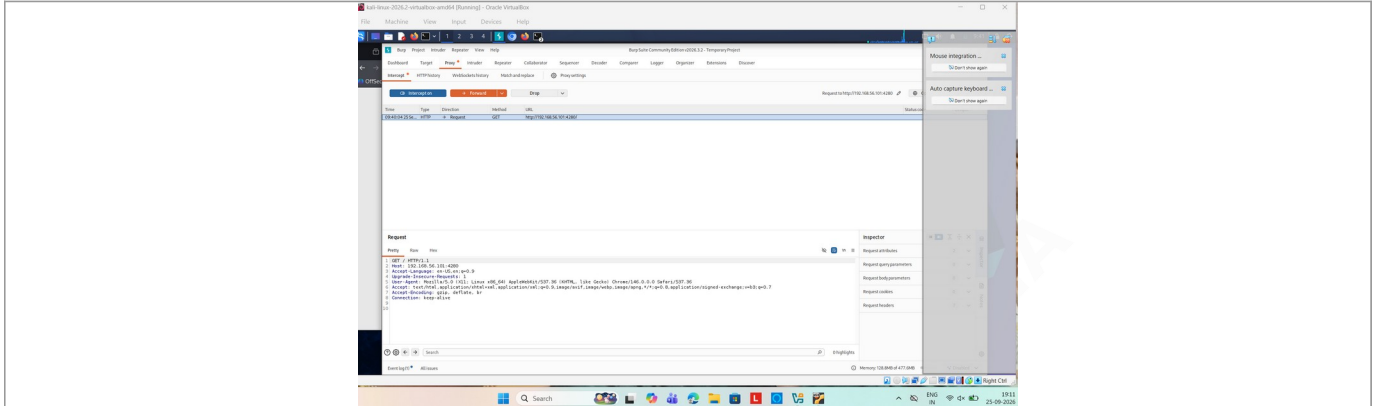


Figure 7: Burp Suite intercepting an HTTP request to the DVWA application during SQL Injection testing.

Reflected XSS Request and Response

The Reflected XSS request was captured and analyzed using Burp Suite's HTTP history. The following details were visible in the captured request and response:

- HTTP method: GET
- Request path: /vulnerabilities/xss_r/
- Host: 192.168.56.101:4280
- Query parameter: name
- XSS payload (URL-encoded in the request): `<script>alert('Reflected XSS')</script>`
- HTTP response status code: 200 OK
- Response headers included Server: Apache/2.4.68 (Debian) and X-Powered-By: PHP/8.5.10

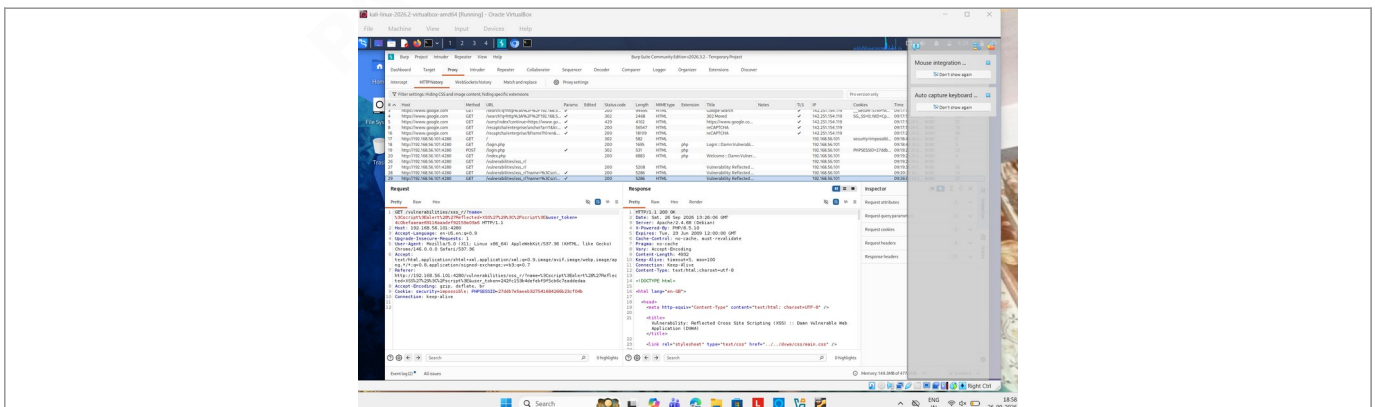


Figure 8: Burp Suite HTTP request and response captured during Reflected XSS testing.

10. OWASP ZAP Automated Scan

OWASP ZAP (Zed Attack Proxy) is an open-source tool used for automated web application vulnerability scanning. It crawls and tests a target application for common security issues and generates a list of alerts describing each finding, along with a risk and confidence rating.

Target: http://192.168.56.101:4280

An automated scan was run against the DVWA target and completed successfully. The ZAP Alerts panel listed several findings, which were reviewed to identify confirmed, non-exaggerated observations. An HTML ZAP report was also generated during the assessment as part of the standard ZAP workflow.

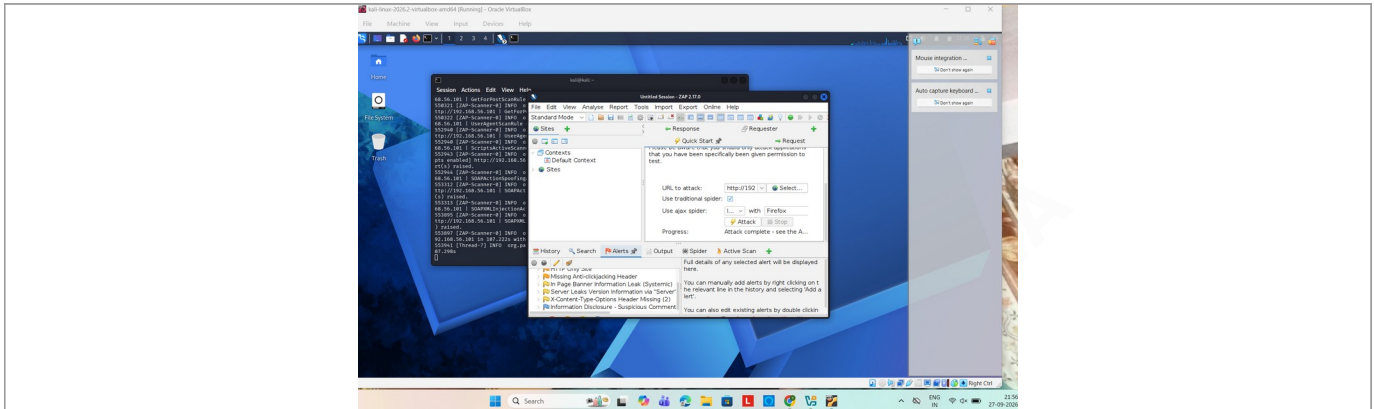


Figure 9: OWASP ZAP Alerts identified during the automated scan.

11. Confirmed ZAP Findings

1. Missing Anti-clickjacking Header

Risk: Low

Confidence: High

Description: The application response does not include appropriate anti-clickjacking protection.

Mitigation: Configure appropriate anti-clickjacking protections such as Content-Security-Policy frame-ancestors and/or X-Frame-Options where appropriate.

2. Server Leaks Version Information via "Server"

Risk: Low

Confidence: High

Description: The HTTP Server response header exposes server software/version information.

Mitigation: Minimize unnecessary server version information exposed through HTTP response headers.

3. X-Content-Type-Options Header Missing

Risk: Low

Confidence: Medium

Description: The application response does not include the X-Content-Type-Options security header.

Mitigation: Add X-Content-Type-Options: nosniff.

Additional Findings Visible in the ZAP Alerts List

The ZAP Alerts panel also showed the following two findings. Confidence values were not clearly specified for these in the available evidence and are therefore left unstated rather than assumed.

4. In Page Banner Information Leak (Systemic): Risk: Low

5. Information Disclosure – Suspicious Comment: Risk: Informational

PAILACHUKKA SUSMITHA

12. Vulnerability Findings Table

Finding	Risk	Confidence	Description	Recommended Mitigation
Missing Anti-clickjacking Header	Low	High	Response lacks anti-clickjacking protection.	Configure Content-Security-Policy frame-ancestors and/or X-Frame-Options.
Server Leaks Version Information via "Server"	Low	High	HTTP Server header exposes server software/version.	Minimize server version information in response headers.
X-Content-Type-Options Header Missing	Low	Medium	Response lacks the X-Content-Type-Options header.	Add X-Content-Type-Options: nosniff.
In Page Banner Information Leak (Systemic)	Low	Not specified	Page banner information leak identified by ZAP.	Review and minimize banner/version information exposed in page content.
Information Disclosure – Suspicious Comment	Informational	Not specified	A suspicious comment was flagged as potential information disclosure.	Review source code comments before deployment; remove unnecessary comments.

13. Risk Level Explanation

High: Potentially severe security impact.

Medium: Meaningful security risk depending on the application and context.

Low: Limited direct impact but should still be addressed.

Informational: Useful security information that may not represent a direct vulnerability.

For this assessment, the confirmed ZAP findings were Low risk or Informational. Their severity has not been exaggerated, and none of the findings identified through ZAP represent a High or Critical risk in this evidence set.

14. Mitigation and Security Recommendations

- Use parameterized SQL queries.
- Use prepared statements.
- Validate input.
- Apply proper output encoding.
- Implement Content Security Policy.
- Implement anti-clickjacking protection.
- Add X-Content-Type-Options: nosniff.
- Minimize unnecessary server version disclosure.
- Follow secure authentication practices.
- Keep software and dependencies updated.
- Apply secure configuration practices.
- Perform regular vulnerability assessments.
- Integrate security testing into development.

15. Challenges and Solutions

1. DVWA/Docker setup and container availability

Solution: Configured Docker Compose and verified the DVWA service.

2. DVWA initially used a restrictive security level

Solution: Changed the security level to Low for controlled demonstrations.

3. OWASP ZAP initially could not reach DVWA after restarting the VMs

Solution: Started the Docker Compose services again on Ubuntu and verified DVWA connectivity from Kali.

4. Burp Suite and ZAP involved proxy configuration

Solution: Configured the tools appropriately and used the required proxy settings.

16. Learning Outcomes

This assessment helped develop practical understanding of:

- Web application security
- SQL Injection
- Reflected XSS
- Stored XSS
- HTTP requests and responses
- Burp Suite
- OWASP ZAP
- Vulnerability scanning
- Risk and confidence levels
- Security headers
- Web application mitigation
- VAPT documentation
- Authorized security testing

17. Conclusion

This week's assessment provided hands-on experience with the fundamentals of web application vulnerability testing. DVWA was deployed in a local authorized lab environment, and SQL Injection was tested and demonstrated using a baseline comparison followed by a payload-based test. Reflected and Stored XSS were both demonstrated using DVWA's dedicated vulnerability pages.

Burp Suite was used to intercept and analyse the HTTP request and response involved in the Reflected XSS test, providing visibility into the exact request path, parameter, payload, and server response. OWASP ZAP was used to perform an automated scan of the DVWA application, and the resulting alerts were reviewed to identify confirmed, non-exaggerated findings — all of which were rated Low risk or Informational.

Appropriate mitigation techniques were identified for each vulnerability class and finding, covering secure coding practices (parameterized queries, output encoding), security headers, and general secure configuration principles.

This practical did not make the DVWA application completely secure — DVWA remains intentionally vulnerable by design, and the Low security level used for this assessment specifically disables its built-in protections. The exercise improved

understanding of basic web application VAPT methodology: baseline testing, payload-based testing, proxy-based traffic analysis, and automated scanning, together with how to document findings professionally and propose realistic mitigations.

PAILACHUKKA SUSMITHA

18. References

- [OWASP Top 10](#)
- [OWASP ZAP](#)
- [DVWA – Damn Vulnerable Web Application](#)
- [PortSwigger Web Security Academy](#)
- [OWASP SQL Injection Prevention Cheat Sheet](#)
- [OWASP Cross Site Scripting Prevention Cheat Sheet](#)

19. Appendix

Key SQL Injection Payload

```
1' OR '1'='1 #
```

Key XSS Payloads

Reflected XSS: `<script>alert('XSS')</script>`
Stored XSS: `<script>alert('Stored XSS')</script>`

Important Lab Commands

```
sudo docker compose up -d  
curl -I http://192.168.56.101:4280  
zaproxy
```

Screenshot List

Figure	Description
1	DVWA security level configured to Low
2	SQL Injection baseline test (User ID = 1)
3	SQL Injection baseline test viewed via Burp Suite browser
4	SQL Injection payload and response (multiple records returned)
5	Reflected XSS successfully demonstrated in DVWA
6	Stored XSS test performed in DVWA
7	Burp Suite intercepting a request during SQL Injection testing
8	Burp Suite HTTP request and response for Reflected XSS
9	OWASP ZAP Alerts from the automated scan

ZAP Finding Summary

Finding	Risk
Missing Anti-clickjacking Header	Low
Server Leaks Version Information via "Server"	Low
X-Content-Type-Options Header Missing	Low
In Page Banner Information Leak (Systemic)	Low
Information Disclosure – Suspicious Comment	Informational